

SLMForge

Milestone 2: Data Layer + Source Adapters

Summer Profile Building Drive 2026
Newton School of Technology

Section 1: Before We Begin

Quick Recap: What is SLMForge?

SLMForge is an end-to-end platform for fine-tuning Small Language Models (SLMs). It takes raw data from multiple sources, builds clean training datasets, runs fine-tuning jobs, evaluates model quality, and serves the trained model via an API. Think of it as a factory pipeline: raw materials go in, a polished model comes out.

In Milestone 1, you built the factory floor: project scaffold, Docker environment, CI pipeline, database schema (SQLAlchemy + Alembic), and the FastAPI skeleton with the Engine API contract. Now in Milestone 2, you are building the **raw material supply chain** - the data layer that feeds everything downstream.

What Will You Build in Milestone 2?

M2 is titled **Data Layer + Source Adapters**. The milestone description says: *Multi-format ingestion, source adapter pattern, source guard*. You will build 5 issues:

Issue	Title	Key Deliverable
#5	Source adapter base class + registry	ABC with iter_records(), concrete adapters, registry
#6	Multi-format ingestion	jsonl/csv/parquet/txt readers, python-magic detection
#7	Dataset Builder + dataset card	Seeded 80/10/10 split, auto-generated card
#8	Source guard + CI regex scan	Block internal data leaks, pre-commit + CI checks
#9	Public dataset prefetch helper	HuggingFace dataset caching, CLI command

Samajhne ke liye ek analogy: Restaurant ka Supply Chain

SLMForge ek restaurant hai. M1 mein tumne kitchen banaya (scaffold), gas connection liya (Docker), health inspector ko bulaya (CI), menu card likha (API contract). Ab M2 mein tum **ration supply chain** bana rahe ho - sabzi mandi se, online delivery se, apne farm se - saari jagah se ration aayega, lekin kitchen mein pahunchne se pehle sab clean, sorted, aur quality-checked hona chahiye. Yehi hai data layer.

Prerequisite: M1 Must Be Complete

Before starting M2, ensure your M1 deliverables are working:

- `docker compose up` starts all services
- `curl localhost:8000/health` returns OK
- `alembic upgrade head` applies migrations
- CI pipeline passes on the main branch
- API contract document exists in `docs/`

Section 2: Source Adapter Base Class + Registry

Issue #5 | Branch: feature/issue-5-source-adapters | Depends on: Issue #1

Why Multiple Source Types?

SLMForge must handle data from very different origins: synthetic data you generate, public datasets from HuggingFace Hub, local files on disk, and (future) internal Newton School datasets. Each source has its own way of reading data. But the rest of the pipeline (ingestion, splitting, training) should not care WHERE the data came from. It just needs records in a consistent shape.

Dabba/Tiffin Analogy: Ek Format, Kahin Se Bhi Aaye

Jaise tumhara tiffin box same shape ka hota hai - chahe mummy ne ghar pe khana bharaa ho, mess se aaya ho, ya Zomato se order kiya ho. Tiffin open karo, roti-sabzi milegi. Source adapter bhi aisa hi hai - chahe data synthetic ho, HuggingFace se ho, ya local CSV se - `iter_records()` call karo, normalised records milenge. Downstream code ko pata hi nahi chalta ki data kahan se aaya.

The Abstract Base Class

The base class defines the contract that every source adapter must follow. It has two methods:

- `iter_records()` - Yields records one by one (lazy iterator, memory-efficient)
- `metadata()` - Returns a dict with source type, path/id, size, licence info

```
# src/slmforge/data/sources/base.py
from abc import ABC, abstractmethod
from typing import Iterator, Dict, Any

class BaseSource(ABC):
    """Abstract base for all data sources."""

    @abstractmethod
    def iter_records(self) -> Iterator[Dict[str, Any]]:
        """Yield normalised records: {"text": ..., "meta": ...}"""
        ...

    @abstractmethod
    def metadata(self) -> Dict[str, Any]:
        """Return source metadata: type, id/path, size, licence."""
        ...
```

Try This

Create `src/slmforge/data/sources/base.py` with the ABC above. Run: `python -c "from slmforge.data.sources.base import BaseSource; print('OK!')"`

Concrete Adapters

Three concrete adapters implement `BaseSource`. Each knows how to read from its source:

SyntheticSource

Generates training data programmatically. Useful for bootstrapping when you have no real data yet. Takes a generator function and count as config.

```

# src/slmforge/data/sources/synthetic.py
from .base import BaseSource
from typing import Iterator, Dict, Any, Callable

class SyntheticSource(BaseSource):
    def __init__(self, generator_fn: Callable, count: int = 1000,
                 name: str = 'synthetic'):
        self._gen = generator_fn
        self._count = count
        self._name = name

    def iter_records(self) -> Iterator[Dict[str, Any]]:
        for i in range(self._count):
            yield self._gen(i)

    def metadata(self) -> Dict[str, Any]:
        return {
            "type": "synthetic",
            "name": self._name,
            "size": self._count,
            "licence": "generated"
        }

```

PublicHFSource

Wraps HuggingFace datasets library. Takes a dataset_id (like 'samsum') and optional split/config.

```

# src/slmforge/data/sources/public.py
from .base import BaseSource
from typing import Iterator, Dict, Any

class PublicHFSource(BaseSource):
    def __init__(self, dataset_id: str, split: str = 'train',
                  text_field: str = 'text'):
        self._dataset_id = dataset_id
        self._split = split
        self._text_field = text_field
        self._ds = None # lazy load

    def _load(self):
        if self._ds is None:
            from datasets import load_dataset
            self._ds = load_dataset(self._dataset_id,
                                    split=self._split)

    def iter_records(self) -> Iterator[Dict[str, Any]]:
        self._load()
        for row in self._ds:
            yield {
                "text": row[self._text_field],
                "meta": {"source": self._dataset_id}
            }

    def metadata(self) -> Dict[str, Any]:
        self._load()
        return {
            "type": "public_hf",
            "id": self._dataset_id,
            "split": self._split,
            "size": len(self._ds),
            "licence": self._ds.info.license or "unknown"
        }

```

LocalSource

Reads data from local files. Delegates actual parsing to the ingest module (Issue #6).

```

# src/slmforge/data/sources/local.py
from .base import BaseSource
from typing import Iterator, Dict, Any
from pathlib import Path

class LocalSource(BaseSource):
    def __init__(self, path: str, text_field: str = 'text'):
        self._path = Path(path)
        self._text_field = text_field

    def iter_records(self) -> Iterator[Dict[str, Any]]:
        from slmforge.data.ingest import load_file
        for record in load_file(self._path):
            yield {
                "text": record[self._text_field],
                "meta": {"path": str(self._path)}
            }

    def metadata(self) -> Dict[str, Any]:
        return {
            "type": "local",
            "path": str(self._path),
            "size": self._path.stat().st_size,
            "licence": "local"
        }

```

InternalSource (Stub)

A placeholder for future Newton School internal datasets. Raises `NotImplementedError` with a clear message so nobody accidentally tries to use it before it is ready.


```

# src/slmforge/data/sources/internal.py
from .base import BaseSource
from typing import Iterator, Dict, Any

class InternalSource(BaseSource):
    """Stub - not yet implemented."""

    def iter_records(self) -> Iterator[Dict[str, Any]]:
        raise NotImplementedError(
            "InternalSource is not yet available. "
            "This adapter will be implemented when NST "
            "internal data pipelines are ready."
        )

    def metadata(self) -> Dict[str, Any]:
        raise NotImplementedError(
            "InternalSource metadata not available."
        )

```

The Registry

The registry is a simple dict mapping source type strings to adapter classes. When you want a source, you ask the registry by type name. If the type is unknown, it raises a clear error.

```
# src/slmforge/data/sources/registry.py
from typing import Dict, Type
from .base import BaseSource
from .synthetic import SyntheticSource
from .public import PublicHFSource
from .local import LocalSource
from .internal import InternalSource

_REGISTRY: Dict[str, Type[BaseSource]] = {
    "synthetic": SyntheticSource,
    "public_hf": PublicHFSource,
    "local": LocalSource,
    "internal": InternalSource,
}

def get_source(source_type: str, **kwargs) -> BaseSource:
    """Factory: get a source adapter by type name."""
    if source_type not in _REGISTRY:
        raise ValueError(
            f"Unknown source type: {source_type}. "
            f"Available: {list(_REGISTRY.keys())}"
        )
    return _REGISTRY[source_type](**kwargs)

def list_source_types() -> list:
    return list(_REGISTRY.keys())
```

Phone Directory Analogy: Registry = Phonebook

Registry ek phone directory hai. Tum naam dete ho ('synthetic', 'public_hf', 'local'), directory tumhe correct class return karti hai. Agar koi unknown naam do ('magic_source'), toh 'number not found' - ValueError. Simple, predictable, extensible. Naya source type add karna hai? Bas ek line add karo registry mein.

Try This

Run: `pytest tests/unit/test_source_registry.py -v`

Verify: (1) `get_source('synthetic', generator_fn=lambda i: {'text': f'record {i}'}, count=5)` returns a `SyntheticSource`. (2) `get_source('unknown_type')` raises `ValueError`. (3) `get_source('internal')` returns `InternalSource` but `iter_records()` raises `NotImplementedError`.

What if we skip this?

Agar adapter pattern skip karo toh har jagah if-else chain lagani padegi: `if source_type == 'csv': ... elif source_type == 'hf': ...` Naye source type add karne pe 10 files change karne padenge. Adapter pattern se sirf registry mein ek line add karo, baaki sab automatic.

Section 3: Multi-Format Ingestion

Issue #6 | Branch: feature/issue-6-multi-format | Depends on: Issue #5

Why Auto-Detect Formats?

Users will throw JSONL, CSV, Parquet, or plain text folders at SLMForge. They should not have to specify the format manually. The system should detect the format automatically and load it into a normalised record stream. This is the 'just works' principle.

Doctor ki Diagnosis: Symptoms Se Bimari Pakadna

Jaise doctor symptoms dekh ke bimari pakad leta hai - bukhar + khansi = flu, bukhar + rash = dengue, bukhar + joint pain = chikungunya. Doctor ko tum explicitly nahi batate ki 'mujhe flu hai', woh khud identify karta hai. python-magic bhi aisa hi hai - file ke first few bytes (magic bytes) dekh ke format pakad leta hai. Extension galat bhi ho toh bytes jhooth nahi bolte.

Supported Formats

Format	Extensi on	Magic Bytes	Reader
JSONL	.jsonl	{"key": ...} per line	json.loads() per line
CSV	.csv	Header row with commas	csv.DictReader
Parquet	.parquet	PAR1 magic header	pyarrow.parquet.read_table
Text Folder	director y/	Directory with .txt files	os.listdir + read each

The Ingest Module

Part 1: Format Detection

```
# src/slmforge/data/ingest.py
import json, csv
from pathlib import Path
from typing import Iterator, Dict, Any, Optional

def detect_format(path: Path) -> str:
    """Detect file format using magic bytes + extension fallback."""
    if path.is_dir():
        return "txt_folder"
    # Read first bytes for magic detection
    with open(path, 'rb') as f:
        header = f.read(16)
    if header[:4] == b'PAR1':
        return "parquet"
    # Extension fallback
    ext = path.suffix.lower()
    if ext == ".jsonl": return "jsonl"
    if ext == ".csv": return "csv"
    if ext == ".parquet": return "parquet"
    raise ValueError(f"Cannot detect format for {path}")
```

Part 2: Per-Format Readers + load_file()

```

# src/slmforge/data/ingest.py (continued)

def _read_jsonl(path: Path) -> Iterator[Dict[str, Any]]:
    with open(path) as f:
        for line in f:
            line = line.strip()
            if line:
                yield json.loads(line)

def _read_csv(path: Path) -> Iterator[Dict[str, Any]]:
    with open(path, newline='') as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield dict(row)

def _read_parquet(path: Path) -> Iterator[Dict[str, Any]]:
    import pyarrow.parquet as pq
    table = pq.read_table(str(path))
    for batch in table.to_batches():
        for row in batch.to_pydict().values():
            yield dict(zip(table.column_names, row))

def _read_txt_folder(path: Path) -> Iterator[Dict[str, Any]]:
    for f in sorted(path.glob('*.txt')):
        yield {"text": f.read_text(), "meta": {"file": f.name}}

_READERS = {
    "jsonl": _read_jsonl, "csv": _read_csv,
    "parquet": _read_parquet, "txt_folder": _read_txt_folder,
}

def load_file(path, fmt: Optional[str] = None):
    """Load records from a file. Auto-detects format."""
    path = Path(path)
    fmt = fmt or detect_format(path)
    if fmt not in _READERS:
        raise ValueError(f"No reader for format: {fmt}")
    return _READERS[fmt](path)

```

Sample Preview Helper

Before building a full dataset, you want to peek at the first few records to verify the data looks correct. The preview helper returns the first N records.

```
# src/slmforge/data/preview.py
from pathlib import Path
from typing import List, Dict, Any
from .ingest import load_file

def preview(path, n: int = 5) -> List[Dict[str, Any]]:
    """Return first n records from a data file."""
    records = []
    for i, record in enumerate(load_file(path)):
        if i >= n:
            break
        records.append(record)
    return records
```

Try This

Create a test JSONL file:

```
echo '{"text":"hello"}\n{"text":"world"}' > /tmp/test.jsonl
```

Then run: `python -c "from slmforge.data.preview import preview; print(preview('/tmp/test.jsonl'))"`

You should see: `[{'text': 'hello'}, {'text': 'world'}]`

What if we skip this?

Agar format detection skip karo toh users ko manually format specify karna padega har baar. Ek galat extension (.csv file jo actually JSONL hai) toh crash. Magic bytes check se ye problem solve hoti hai - extension se zyada reliable.

Section 4: Dataset Builder with Seeded Split

Issue #7 | Branch: feature/issue-7-dataset-builder | Depends on: Issue #6

Why Deterministic Splits?

When you split data into train/validation/eval sets, the split must be **reproducible**. If you run the same code with the same seed tomorrow, you must get the exact same split. This is critical for:

- **Fair comparison** - Two models trained on the same train set can be compared
- **No eval contamination** - Eval records must NEVER appear in training data
- **Debugging** - If a model performs badly, you can inspect the exact training data

Khana Baatna: 80-10-10 Rule

Ghar mein 100 laddoo aaye. 80 laddoo family ke liye (train), 10 laddoo guests ke liye (validation), 10 laddoo fridge mein rakho kal ke liye (eval). Ab important baat: **seed = recipe**. Agar same seed (42) do toh har baar same 80-10-10 division hoga. Wahi laddoo same groups mein jayenge. Agar seed change karo (43), different division hoga. Lekin same seed = same split, guaranteed.

The DatasetBuilder


```

# src/slmforge/data/builder.py
from typing import List
from datasets import Dataset, DatasetDict
from .sources.base import BaseSource

DEFAULT_SEED = 42
DEFAULT_SPLIT = (0.8, 0.1, 0.1) # train, val, eval

class DatasetBuilder:

    @staticmethod
    def build(
        sources: List[BaseSource],
        seed: int = DEFAULT_SEED,
        split_ratio: tuple = DEFAULT_SPLIT
    ) -> DatasetDict:
        """
        Collect records from all sources, shuffle with seed,
        split into train/val/eval.
        """
        # 1. Collect all records
        all_records = []
        for src in sources:
            for record in src.iter_records():
                all_records.append(record)

        # 2. Create a HuggingFace Dataset
        ds = Dataset.from_list(all_records)

        # 3. Shuffle with seed
        ds = ds.shuffle(seed=seed)

        # 4. Split
        train_ratio, val_ratio, eval_ratio = split_ratio
        n = len(ds)
        n_train = int(n * train_ratio)
        n_val = int(n * val_ratio)

        return DatasetDict({
            "train": ds.select(range(0, n_train)),
            "val": ds.select(range(n_train, n_train + n_val)),
            "eval": ds.select(range(n_train + n_val, n)),
        })

```

Dataset Card Auto-Generation

Every dataset you build gets a dataset card - a markdown file documenting what went into it. This is the 'ingredient label' for your data.

```
# src/slmforge/data/card.py
from typing import List
from .sources.base import BaseSource
from datasets import DatasetDict

def generate_card(
    sources: List[BaseSource],
    dataset: DatasetDict,
    seed: int,
    output_path: str = 'data/dataset_card.md'
) -> str:
    """Generate a dataset card markdown file."""
    lines = ['# Dataset Card', '']
    lines.append('## Sources')
    for src in sources:
        meta = src.metadata()
        lines.append(f"- **{meta.get('type', 'unknown')}**: "
                    f"{meta.get('id', meta.get('path', 'N/A'))} "
                    f"(size: {meta.get('size', '?')}), "
                    f"licence: {meta.get('licence', '?')})")
    lines.append('')
    lines.append('## Splits')
    for split_name, split_ds in dataset.items():
        lines.append(f"- **{split_name}**: {len(split_ds)} records")
    lines.append('')
    lines.append(f'## Reproducibility')
    lines.append(f"- Seed: {seed}")
    lines.append(f"- Split ratio: 80/10/10")

    card_text = '\n'.join(lines)
    with open(output_path, 'w') as f:
        f.write(card_text)
    return card_text
```

Product ka Label: Dataset Card

Jaise Maggi ke packet pe likha hota hai - ingredients (maida, palm oil, MSG), weight (70g), manufacturer (Nestle), expiry date. Dataset card bhi aisa hi hai - source (HuggingFace samsun, local CSV), size (10,000 records), licence (Apache 2.0), split ratio (80/10/10), seed (42). Koi bhi dekh ke samajh sakta hai ki dataset mein kya hai.

Try This

Run: `pytest tests/unit/test_dataset_builder.py -v`

Verify: (1) Build with seed=42, note the first 3 record IDs in eval. (2) Build again with seed=42 - same 3 records in eval. (3) Build with seed=99 - DIFFERENT records in eval. (4) Check: no record appears in both train and eval (no leak).

What if we skip this?

Bina deterministic splits ke, har training run pe model alag data se train hoga. Tum compare hi nahi kar paoge ki model A better hai ya B. Aur worst case: eval data training mein leak ho jayega - model 'cheat' karega, real-world performance bahut kharab hogi.

Section 5: Source Guard + CI Regex Scan

Issue #8 | Branch: feature/issue-8-source-guard | Depends on: Issue #5

Why Data Protection?

Newton School has internal data that must **never** enter the public GitHub repo. This is a two-layer defence: a runtime guard (source_guard) and a CI/pre-commit scan.

Do Checkpoints: Gate Security + CCTV

Jaise society mein do level ki security hoti hai: (1) **Entry gate pe guard** (pre-commit hook) - har visitor ko check karta hai BEFORE entry. Agar ID nahi hai, entry nahi milegi. (2) **CCTV cameras** (CI regex scan) - agar koi somehow andar aa gaya, CCTV pakad lega. Dono chahiye - gate guard fast hai (local), CCTV thorough hai (cloud CI). Source guard = gate guard (rejects unregistered types + internal paths). CI regex scan = CCTV (greps for NST identifier patterns in tracked files).

The Source Guard Module

```

# src/slmforge/data/source_guard.py
from pathlib import Path
from .sources.registry import list_source_types

# Paths that must never be used as data sources
BLOCKED_PATHS = ['_internal/', 'internal/', 'nst_data/']

def validate(source_type: str, path: str = None) -> bool:
    """
    Validate a data source before use.
    Raises ValueError on violations.
    """
    # 1. Check: is the source type registered?
    known = list_source_types()
    if source_type not in known:
        raise ValueError(
            f"Unregistered source type: {source_type}. "
            f"Known types: {known}"
        )

    # 2. Check: does the path contain blocked patterns?
    if path:
        for blocked in BLOCKED_PATHS:
            if blocked in str(path):
                raise ValueError(
                    f"Blocked path detected: {path} "
                    f"contains '{blocked}'. Internal data "
                    f"must not enter the public repo."
                )

    return True

```

CI Regex Scan

The CI step greps all tracked files for known NST identifier patterns (internal URLs, internal dataset IDs, specific file paths). If any match is found, the CI job fails with a clear error message.

```
# .github/workflows/ci.yml (add this job)
source-guard:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Scan for internal data leaks
      run: |
        echo 'Scanning for NST internal identifiers...'
        PATTERNS='_internal/|nst_data/|internal\.nst\.'
        if git grep -E "$PATTERNS" -- '*.py' '*.yaml' '*.json'; then
          echo '::error::Internal data pattern detected!'
          exit 1
        fi
        echo 'Clean - no internal data patterns found.'
```

Pre-Commit Hook

The pre-commit hook runs the same check locally before you can even commit. This catches violations immediately, before they reach CI.

```
# .pre-commit-config.yaml (add this hook)
repos:
  - repo: local
    hooks:
      - id: source-guard
        name: Source guard scan
        entry: bash -c 'grep -rE "_internal/|nst_data/" --include="*.py" . && exit 1 || exit 0'
        language: system
        pass_filenames: false
```

Airport Security Analogy: Pre-Commit = Boarding Gate, CI = Immigration

Pre-commit hook = boarding gate pe bag check. Fast hai, basic contraband pakad leta hai. Lekin sab kuch nahi check hota. CI regex scan = immigration counter with full passport scan. Thorough hai, sab check hota hai. Dono chahiye - gate check se 90% violations wahi ruk jayenge (fast feedback), immigration se remaining 10% pakde jayenge (thorough scan).

Try This

Test the guard:

```
python -c "from slmforge.data.source_guard import validate;
validate('synthetic')" (should pass)
python -c "from slmforge.data.source_guard import validate;
validate('magic')" (should raise ValueError)
python -c "from slmforge.data.source_guard import validate;
validate('local', '_internal/data.csv')" (should raise ValueError)
```

What if we skip this?

Bina source guard ke, ek galti se internal data push ho sakta hai public repo mein. Ye security breach hai - data leak. Pre-commit + CI dono milke ensure karte hain ki internal data kabhi public repo mein na jaaye. Two layers of defence.

Section 6: Public Dataset Prefetch Helper

Issue #9 | Branch: feature/issue-9-prefetch | Depends on: Issue #5

Why Prefetch?

Training a model can take hours. You do not want it to fail 30 minutes in because a HuggingFace dataset download timed out. Prefetch downloads and caches the dataset **before** training starts. If the dataset is already cached, the prefetch is a no-op (instant).

Ration Ki Advance Kharidari

Diwali se pehle sab ration store kar lete hain - atta, cheeni, ghee, mewa. Diwali ke din market jaake dhundhna shuru karo toh bahut late ho jaoge. Prefetch bhi aisa hi hai - training shuru hone se pehle saara dataset download + cache kar lo. Dobara run karo? Already cached hai, skip! Jaise ration already store mein pada hai.

The Prefetch Module


```

# src/slmforge/data/prefetch.py
from pathlib import Path
from datasets import load_dataset
import logging

logger = logging.getLogger(__name__)
CACHE_DIR = Path('data/cache')

def prefetch(
    dataset_id: str,
    split: str = 'train',
    cache_dir: Path = CACHE_DIR
) -> Path:
    """
    Download + cache a HuggingFace dataset.
    Returns the cache path. No-op if already cached.
    """
    cache_path = cache_dir / dataset_id.replace('/', '_')
    cache_path.mkdir(parents=True, exist_ok=True)

    marker = cache_path / '.done'
    if marker.exists():
        logger.info(f'{dataset_id} already cached at {cache_path}')
        return cache_path

    logger.info(f'Downloading {dataset_id}...')
    ds = load_dataset(dataset_id, split=split,
                      cache_dir=str(cache_path))

    # Write licence info to dataset card
    licence = ds.info.license or 'unknown'
    card_path = cache_path / 'LICENCE_INFO.md'
    card_path.write_text(
        f'# {dataset_id}\n'
        f'- Licence: {licence}\n'
        f'- Size: {len(ds)} records\n'
        f'- Split: {split}\n'
    )

    marker.touch() # Mark as done
    logger.info(f'Cached {dataset_id} at {cache_path}')
    return cache_path

```

CLI Integration

The prefetch command is exposed via the SLMForge CLI. Users can run it from the terminal:

```
# Extend src/slmforge/cli/main.py
import click
from slmforge.data.prefetch import prefetch

@click.group()
def cli():
    pass

@cli.group()
def data():
    """Data management commands."""
    pass

@data.command()
@click.argument('dataset_id')
@click.option('--split', default='train')
def prefetch_cmd(dataset_id, split):
    """Download + cache a public dataset."""
    path = prefetch(dataset_id, split=split)
    click.echo(f'Cached at: {path}')

# Usage: slmforge data prefetch samsum
```

Try This

Run: `slmforge data prefetch samsum`

Then check: `ls data/cache/samsum/` - should see cached files + LICENCE_INFO.md

Run again: `slmforge data prefetch samsum` - should print 'already cached' instantly.

What if we skip this?

Bina prefetch ke, training job start hoga, aadhe ghante baad download fail hoga (network timeout), saara compute waste. Prefetch ensures ki data ready hai BEFORE training starts. Aur licence info track karna important hai for legal compliance.

Section 7: Putting It All Together

Data Layer Folder Structure

```
src/slmforge/data/  
sources/  
  __init__.py  
  base.py          # BaseSource ABC  
  synthetic.py     # SyntheticSource adapter  
  public.py       # PublicHFSource adapter  
  local.py        # LocalSource adapter  
  internal.py     # InternalSource stub  
  registry.py     # type -> adapter mapping  
  ingest.py       # Multi-format readers + detection  
  preview.py      # Sample preview helper  
  builder.py      # DatasetBuilder with seeded splits  
  card.py         # Dataset card auto-generation  
  source_guard.py  # Runtime validation  
  prefetch.py     # HF dataset caching
```

Itne Saare Files Kyun? Hospital Departments Analogy

Jaise hospital mein alag-alag departments hote hain: Reception (registry.py - patients ko sahi department bhejta hai), OPD (base.py - common examination protocol), Pathology Lab (ingest.py - samples process karta hai), Pharmacy (builder.py - sahi dose mein dawai baatna/split karna), Security (source_guard.py - kaun andar aa sakta hai kaun nahi), Medical Store (prefetch.py - advance mein dawai stock karna). Har department ka apna kaam hai, lekin saath milke poora hospital chalta hai.

The Data Pipeline Flow

The complete flow from raw data to training-ready dataset:

- **Step 1 - Source Selection:** Choose source type (synthetic, public_hf, local). Registry returns the correct adapter.
- **Step 2 - Source Guard:** Validate the source type and path. Block unregistered types and internal paths.

- **Step 3 - Prefetch (if public):** Download and cache the dataset. No-op if already cached.
- **Step 4 - Ingestion:** Read records through the adapter. Auto-detect format for local files.
- **Step 5 - Preview:** Peek at first 5 records to verify data shape.
- **Step 6 - Build Dataset:** Collect all records, shuffle with seed, split 80/10/10.
- **Step 7 - Generate Card:** Write dataset card with sources, sizes, licences, seed.

Dependency Chain

Issue	Depends On	Blocked By	Reason
#5 Source Adapters	#1 (M1 scaffold)	None	ABC + registry are foundation
#6 Multi-Format	#5	Needs BaseSource	LocalSource delegates to ingest
#7 Dataset Builder	#6	Needs adapters + ingest	build() collects from sources
#8 Source Guard	#5	Needs registry	validate() checks registered types
#9 Prefetch	#5	Needs BaseSource	Caches for PublicHFSource

Section 8: Action Plan

Bina M2 ke? Shaadi Ka Khana Without Ration

M1 mein tumne kitchen banaya (scaffold), gas connection liya (Docker), health inspector bulaya (CI). Ab imagine karo shaadi ka khana banana hai lekin ration hi nahi hai: sabzi mandi = source adapters (kahan se aayega), ration sorting = ingest (clean karna), baatna = builder (80% guests, 10% family, 10% fridge), security guard = source guard (galat cheez andar na aaye), advance kharidari = prefetch (pehle se stock karna). Bina in sab ke, kitchen ready hai lekin kuch ban nahi sakta!

5-Day Breakdown

Day	Focus	Issue	Verify
Day 1	Source adapter ABC + registry	#5	pytest test_source_registry.py passes. get_source('synthetic') works. get_source('unknown') raises ValueError.
Day 2	Multi-format ingestion	#6	pytest test_ingest.py passes. JSONL, CSV, Parquet, txt-folder all load. preview() returns first 5 records.
Day 3	Dataset Builder + card	#7	pytest test_dataset_builder.py passes. Same seed = same split. Dataset card generated.
Day 4	Source guard + CI scan	#8	pytest test_source_guard.py passes. CI fails on deliberate violation. pre-commit rejects internal paths.

Day 5	Prefetch + integration test	#9	slmforge data prefetch samsum works. Subsequent run is instant no-op. End-to-end pipeline test passes.
----------	--------------------------------	----	--

Git Workflow

Each issue gets its own feature branch. Work in order (Issue 5 first, then 6, etc.) because of the dependency chain.

Issue	Branch Name	Base
#5	feature/issue-5-source-adapters	main
#6	feature/issue-6-multi-format	main (after #5 merged)
#7	feature/issue-7-dataset-builder	main (after #6 merged)
#8	feature/issue-8-source-guard	main (after #5 merged)
#9	feature/issue-9-prefetch	main (after #5 merged)

Note: Issues #8 and #9 only depend on #5, not on #6 or #7. So after #5 is merged, you can work on #6, #8, and #9 in parallel if you have multiple team members.

Section 9: Defense Questions

Faculty will ask these during your milestone review. Prepare answers.

Q1: Walk through the source adapter pattern. Why not just if-else?

The adapter pattern (ABC + registry) follows the Open/Closed Principle: open for extension, closed for modification. To add a new source type, you create one new file and add one line to the registry. With if-else, you would need to modify every function that handles sources. The adapter pattern also enforces a contract (iter_records + metadata) that guarantees all sources produce data in the same shape.

Q2: How does format detection work? What if the extension is wrong?

Format detection uses a two-step approach: first check magic bytes (first few bytes of the file), then fall back to file extension. Magic bytes are more reliable because they come from the actual file content. A .csv file that is actually JSONL will be correctly detected as JSONL by the magic bytes check. For Parquet, the magic bytes are 'PAR1' at the start of the file.

Q3: What happens if the same record appears in both train and eval?

This is called 'data leakage' or 'eval contamination'. The model sees the eval record during training, so its eval performance is artificially inflated. In production, it would perform worse. Our DatasetBuilder prevents this by using a single shuffle-then-split approach: records are first shuffled (deterministically via seed), then sliced into contiguous ranges. No record can appear in two ranges.

Q4: Why two layers of data protection (source guard + CI)?

Defence in depth. Pre-commit is fast but local (can be skipped with --no-verify). CI runs in the cloud on every PR and cannot be bypassed. The runtime source_guard catches violations at code execution time. Together, they cover three attack vectors: accidental commit, bypassed pre-commit, and runtime misuse.

Q5: Why cache datasets? What about disk space?

Training jobs can take hours. A mid-training download failure wastes all compute time. Caching ensures data is available before training starts. The .done marker file makes subsequent prefetches instant (no-op). Disk space is managed by the cache directory -

you can clear it manually or add a TTL-based eviction in a future milestone.

Section 10: Glossary

Term	Meaning
ABC (Abstract Base Class)	Python class that defines a contract. Cannot be instantiated directly. Subclasses must implement all abstract methods.
Adapter Pattern	Design pattern where different implementations share the same interface. Client code works with the interface, not the concrete class.
Registry Pattern	A mapping (dict) from keys to classes/functions. Factory for creating objects by name.
Magic Bytes	The first few bytes of a file that identify its format. More reliable than file extension.
python-magic	Python library that wraps libmagic to detect file types from magic bytes.
JSONL	JSON Lines: one JSON object per line. Efficient for streaming large datasets.
Parquet	Columnar binary format by Apache. Efficient for large tabular data. Compressed.
DatasetDict	HuggingFace datasets container holding named splits (train, val, eval).
Seed	Integer that initializes the random number generator. Same seed = same random sequence = reproducible results.
Data Leakage	When evaluation data accidentally appears in training data. Inflates metrics, hurts real-world performance.

Dataset Card	Markdown document describing a dataset: sources, sizes, licences, split ratios.
Source Guard	Runtime validation that blocks unregistered source types and internal data paths.
Pre-commit Hook	Script that runs before git commit. Catches violations early (local, fast).
CI Regex Scan	Continuous Integration step that greps code for forbidden patterns. Runs on every PR (cloud, thorough).
Prefetch	Download + cache data before it is needed. Prevents mid-pipeline download failures.
No-op	No operation: function call that does nothing because work is already done (e.g., data already cached).
Click (library)	Python library for building CLI applications. Used for slmforge CLI commands.
Defence in Depth	Security principle: multiple layers of protection. If one layer fails, others catch the issue.